Aarhus University

Department Of Computer Science

Compilation

Phase 2

Author
Søren M. Damsgaard

Student number
202309814

October 12, 2026



Overview of phase 2

The point of this phase is to implement `ForStm`, `WhileStm`, `BreakStm`, `ContinueStm` and the support of multiple variable declarations, into the semantics and code generation, while extending the test environment to include the newly added code.

Task 1 - Semantic Analysis

To support multiple variables `VarDeclStm` was refactored, the original code made into a helper function `typecheck_single_declaration`, the new code calls another helper function `typecheck_decl_block` that uses foldleft to iterate over a list on declarations applying `typecheck_single_declaration` on them.

For the loops, a bool flag was added to the `Env` to ascertain wether we are in a loop or not, the `BreakStm` and `ContinueStm` use this for validity.
The `WhileStm` force the condition to evalaute to a bool, while the body is evaluated in a loop env where the loop flag is flipped, the new environment is discarded after evaluation to avoid side-effects from the loop, and to reverse the flag.
The `ForStm` does a lot of the same, but it does so inside a new scope to allow variable declarations in the 'init' statement, and it uses the 'Option' type to allow the condition and update statements to be optional.

Task 2 - Code generation

The code generation of `VarDeclStm` follows the same form refactoring that was made in the semantic part, move orignal code and use helepr functions with foldleft.

For the loop part a new tuple of type 'option' was added to `cgenv` containing the jump label for continue and break, this means they can be none whenever we aren't in a loop.
The `BreakStm` and `ContinueStm` use this as their flag, if the the labels are 'none' it fails.
For the `WhileStm` and `ForStm`, the logic remains the same as in the semantic analysis, in regards to optional statement and having closed scopes for the loops.

Task 3 - Tests

The test environment has been refactored, so it uses the `OUnit2` library, and tests now exists in separate folders, respective of their phase in which they were implemented.

Individual Contribution

A lot of the coding was done together on the same computer, by taking turns coding cases in both the semantic and codegen files, and for the testing we both implemented tests from a testlist covering most of the new code.

How to run the code

You will have received a zip file containing the complete OCaml compiler source code, the Dune build configurations, and the C runtime environment.

This is run/compiled in VScode using the docker container provided.
To compile and run the project inside a VS Code development container, execute the following commands in the terminal while inside the folder:
The following commands is to run a test file manually

1. `dune build`
2. `dune exec dolphin -- compile --from-ast --phase 2 tests/phase2/[name of test].json -o out.ll` (This runs and compiles a test, and outputs it into a file out.ll)
3. `clang runtime.c out.ll -o ret`
4. `./ret` (outputs any prints into the terminal, like 'print_integer()')
5. `echo $?` (Outputs the result of the test)

The following commands is to run all the test suits using `OUnit2`

1. `dune build` (if not already doen)
2. `dune test` (runs the 'test_dolphin.ml' file)

The test file asserts each test case with the expected output, be it error messages or numerical output.